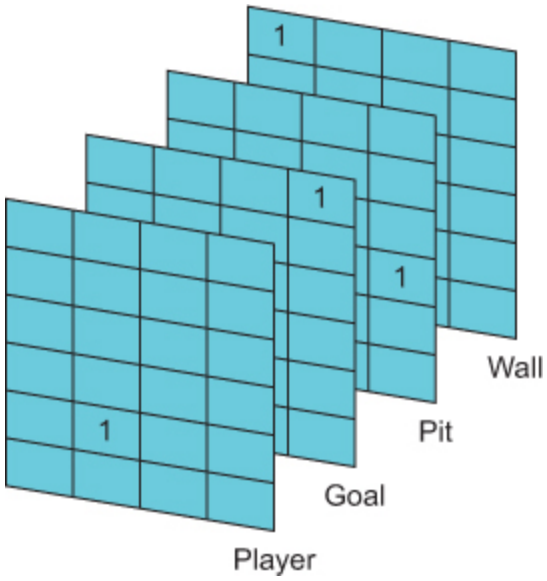
the game state, you can see that the first matrix encodes the position of the player, the second matrix encodes the position of the goal, the third matrix encodes the position of the pit, and the last matrix encodes the position of the wall.

In other words, the first dimension of this 3-tensor is divided into four separate grid planes, where each plane represents the position of each element. Figure 3.7 shows an example where the player is at grid position (2,2), the goal is at (0,0), the pit is at (0,1), and the wall is at (1,1), where the planes are (row, column). All other elements are 0s.

Figure 3.7. This is how the Gridworld board is represented as a numpy array. It is a 4 x 4 x 4 tensor, composed of 4 "slices" of a 4 x 4 grid. Each grid slice represents the position of an individual object on the board and contains a single 1, with all other elements being 0s. The position of the 1 indicates the position of that slice's object.



While we could, in principle, build a neural network that can operate on a 4 × 4 × 4 tensor, it is easier to just flatten it into a 1-tensor (a vector). A 4 × 4 × 4 tensor has $4^3 = 64$ total elements, so the input layer of our neural network must be accordingly shaped. The neural network will have to learn what this data means and how it relates to maximizing rewards. Remember, the algorithm will know absolutely nothing to begin with.

3.2.7. A neural network as the Q function